UIPATH · CODED AGENT

Amplify — how it works

When a feature ships, Amplify turns it into sales-ready enablement — a one-pager and a video — grounded in the actual pull request, on-brand, and dropped straight into Data Service. This is the internal walk-through of every part.

Trigger Jira label `Amplify`

Brain Claude Opus 5

Runtime UiPath coded agent (LangGraph)

Output `EnablementAsset` + Storage Bucket

01 The shape of it

Four phases, one graph. Capture the context while it's fresh, let two directors author the story, produce the assets, and gate everything before anything is stored.

02 The trigger

Enablement fails when someone reconstructs context weeks later. So the trigger sits at the moment of maximum context: the ticket.

JIRA LABEL

Amplify

Add the label `Amplify` to a Jira issue and an Integration Service trigger starts the coded agent with the issue key. The agent reads the ticket, pulls the **linked pull-request URLs** from it, and hands them to ingest. No form, no copy-paste — labelling the ticket is the whole action.

`Jira issue + label Amplify` → `IS trigger → start agent` → `read issue → PR links` → `ingest`

03 Inside the graph

The agent is a LangGraph `StateGraph`: `ingest → author → render → store`. Each node is small and single-purpose; state flows between them. Here is what each one really does.

04 Where it runs

Two runtimes, on purpose. The agent is serverless; the frame-render lives on a worker with a browser — because headless-Chrome rendering can't run inside a serverless sandbox.

Why split? A one-pager is text, so it renders in the agent and the release path is fully automatic. A video is a browser seeking a timeline frame by frame — that needs Chrome, which the serverless runtime doesn't host. Putting the frame-render on a worker keeps the agent light and the video real.

05 What gets stored

One row per asset in the `EnablementAsset` Data Service entity. Field names are camelCase; URLs are durable `bucket://` references the App resolves on demand.

FIELD	TYPE	WHAT IT HOLDS
`assetType`	`string`	feature · release
`title`	`string`	The feature / release name
`product`	`string`	Concise product line — StudioWeb, Verticals, Maestro…
`description`	`string`	One-line "what it's about" shown under the title
`spoc`	`string`	Single point of contact — PR author / assignee
`sourceRef`	`string`	The PR / release URL it was generated from
`videoUrl`	`string`	`bucket://` reference to the MP4
`onepagerUrl`	`string`	`bucket://` reference to the one-pager
`digestUrl`	`string`	`bucket://` reference to the release digest
`qaStatus`	`string`	Render-QA result — passed · flags
`claimCheckStatus`	`string`	Claim-check result — reviewed · flags
`reviewStatus`	`boolean`	Human approval — **false** on generation

06 Built on

Two directors are the only things built from scratch; the platform provides the rest and Amplify configures it.

UiPath Agents SDK
`uipath · uipath-langchain`

LangGraph
`StateGraph orchestration`

Claude Opus 5
`AI Trust Layer · Bedrock`

Data Service
`EnablementAsset entity`

Storage Buckets
`amplify-assets`

Orchestrator assets
`GitHub PAT (credential)`

Integration Service
`Jira · GitHub`

HyperFrames
`headless Chrome + FFmpeg`

GSAP
`authored scene motion`

ElevenLabs · Whisper
`voice · captions`

`Amplify · from merged to market, automatically.`

Merge is the capture trigger, not the publish trigger — the moment of maximum context, caught before it evaporates.